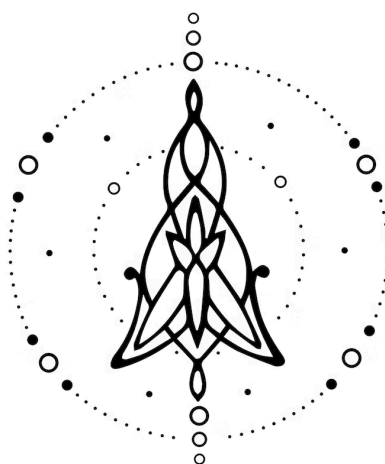




Especificación

Diseño e Implementación de un Lenguaje

8 de Marzo de 2026

Equipo	1
Repositorio	1
Dominio	2
Construcciones	3
Casos de prueba	4
Casos de aceptación	4
Casos de rechazo	5
Ejemplos	5

Equipo

Nombre	Apellido	Legajo	E-mail
Joaquín	Nolasco de Carlés	65368	jnolascodecarles@itba.edu.ar
Nicanor	Novotny	65530	nnovotny@itba.edu.ar
Agustín	Korman	65116	agkorman@itba.edu.ar

Repositorio

La solución será versionada en: <https://github.com/agkorman/Flex-Bison-Compiler>

Dominio

El dominio del lenguaje StackForge es la especificación declarativa de arquitecturas de aplicaciones multi contenedor para entornos de desarrollo y prueba. En este dominio, el interés principal no está en la configuración manual de cada detalle del archivo de despliegue, sino en la descripción de los componentes lógicos de una aplicación y de las relaciones existentes entre ellos.

Los conceptos fundamentales del dominio son los servicios según su rol, por ejemplo:

- frontend: para representar la capa de presentación.
- api : para representar la lógica de negocio o capa de servicios(api layer).
- database: para representar el almacenamiento.
- cache: para representar el almacenamiento caché , o temporal..

Además de estos roles, también forman parte del dominio las relaciones entre componentes, como la exposición externa de un servicio y la conectividad interna entre dos servicios. De esta manera, el lenguaje permite modelar aplicaciones típicas de dos o tres capas, así como configuraciones más complejas que incluyan cosas como caché u otros servicios auxiliares.

El problema que se busca resolver es que, al describir aplicaciones multicontenedor mediante herramientas de despliegue tradicionales, el programador debe escribir explícitamente muchos detalles estructurales, incluso cuando la arquitectura deseada es simple y responde a patrones conocidos. StackForge propone elevar el nivel de abstracción.El usuario describe qué rol cumple cada servicio y cómo se relaciona con los demás, mientras que el compilador se encarga de inferir parte de la configuración final necesaria.

En este sentido, el lenguaje no se limita a reexpresar una especificación existente con otra sintaxis, sino que busca capturar directamente la estructura conceptual de una arquitectura multi contenedor simple.

Construcciones

El lenguaje **StackForge** ofrecerá las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrá crear una aplicación mediante un bloque principal identificado con un nombre.
- (II). Se podrán declarar uno o varios servicios de tipo frontend, api, database o caché, cada uno con un nombre único e imagen base asociada.
- (III). Cada rol representará una abstracción propia del dominio, permitiendo modelar directamente componentes frecuentes de una arquitectura multi contenedor.
- (IV). Se podrá exponer externamente un servicio mediante una construcción específica, indicando el puerto del host a través del cual será accesible.
- (V). Se podrá declarar conectividad entre dos servicios mediante una construcción relacional, permitiendo expresar que uno necesita comunicarse con otro.
- (VI). El compilador podrá inferir dependencias lógicas entre servicios a partir de las relaciones de conectividad definidas en el programa.
- (VII). El compilador podrá inferir redes internas de comunicación entre servicios, sin necesidad de que el usuario las declare manualmente.
- (VIII). Los servicios de tipo database podrán implicar automáticamente persistencia básica, mediante la generación de volúmenes o estructuras equivalentes en el artefacto final.
- (IX). Los servicios de tipo database y caché se considerarán, por defecto, internos a la arquitectura y no expuestos hacia el exterior.
- (X). El lenguaje contará con verificaciones semánticas mínimas, tales como unicidad de nombres de servicio, existencia de referencias en relaciones connects_to, validez de puertos expuestos y obligatoriedad de los datos mínimos requeridos para cada servicio.
- (XI). A partir de una especificación válida, el compilador generará automáticamente un archivo docker-compose.yml consistente con la arquitectura declarada por el usuario.
- (XII). El lenguaje priorizará una sintaxis compacta y declarativa, evitando exponer al programador todos los detalles estructurales del archivo de salida cuando estos puedan deducirse automáticamente a partir del rol y de las relaciones definidas.

Casos de prueba

Se proponen los siguientes casos iniciales de prueba de aceptación para el lenguaje, orientado a la definición declarativa de aplicaciones multicontenedor a partir de roles de servicio como frontend, api, database y cache. En esta propuesta, el compilador infiere automáticamente parte de la configuración de despliegue, como redes internas, dependencias y persistencia básica, reduciendo la complejidad de la especificación escrita por el usuario. Esta decisión se alinea con la idea de abstraer el dominio mediante construcciones de más alto nivel, en lugar de exponer directamente todos los detalles del artefacto final.

Casos de aceptación

- (I). Un programa que declare una aplicación con un único servicio de tipo frontend.
- (II). Un programa que declare una aplicación con un único servicio de tipo api.
- (III). Un programa que declare una aplicación con un único servicio de tipo database, permitiendo que el compilador infiera persistencia básica.
- (IV). Un programa que declare un frontend expuesto en un puerto del host.
- (V). Un programa que declare un api expuesto en un puerto del host.
- (VI). Un programa que establezca una relación de conectividad entre un servicio api y un servicio database.
- (VII). Un programa que establezca una relación de conectividad entre un frontend y un api.
- (VIII). Un programa que modele una arquitectura compuesta por frontend, api y database, permitiendo que el compilador infiera redes y dependencias.
- (IX). Un programa que declare un servicio de tipo cache y lo conecte a un api.
- (X). Un programa que modele una aplicación con frontend, api, database y cache, incluyendo exposición externa solo de los servicios necesarios.
- (XI). Un programa que declare múltiples servicios del mismo rol con nombres distintos, por ejemplo dos APIs independientes.
- (XII). Un programa que combine roles, exposición de puertos y relaciones de conectividad en una única especificación compacta.

Casos de rechazo

Además, se proponen los siguientes casos iniciales de prueba de rechazo:

- (I). Un programa que defina dos servicios con el mismo nombre.
- (II). Un programa que intente exponer un servicio en un puerto inválido.
- (III). Un programa que establezca una relación `connects_to` hacia un servicio inexistente.
- (IV). Un programa que utilice un rol inexistente o no soportado por el lenguaje.
- (V). Un programa que declare un servicio sin especificar la imagen base correspondiente.
- (VI). Un programa que intente exponer externamente un servicio database cuando dicha operación no esté permitida por las reglas del lenguaje.
- (VII). Un programa malformado sintácticamente, por ejemplo con bloques incompletos o delimitadores ausentes.

Ejemplos

En el Cód. (6.1), se puede ver la definición de una aplicación simple compuesta por un único servicio de tipo frontend. El servicio utiliza una imagen de Nginx y se expone en el puerto 8080 del host. Este ejemplo ilustra una de las ideas principales del lenguaje: permitir que el programador describa la intención arquitectónica mediante un DSL de alto nivel, en lugar de configurar manualmente todos los detalles estructurales.

```
app MiAplicacion {  
  frontend web using "nginx:latest";  
  expose web on 8080;  
}
```

Código 6.1: Definición de una aplicación con un servicio frontend expuesto.

En el Cód. (6.2), se modela una aplicación compuesta por una API y una base de datos. El servicio api se expone hacia el exterior, mientras que el servicio database permanece interno. La relación `connects_to` permite expresar de forma declarativa que la API necesita comunicarse con la base de datos, dejando al compilador la inferencia de dependencias, redes internas y configuración asociada.

```
app MiAplicacion {  
  api backend using "node:20";  
  database db using "postgres:16";  
  
  expose backend on 3000;  
  backend connects_to db;  
}
```

Código 6.2: Aplicación con API y base de datos mediante conectividad declarativa.

En el Cód. (6.3), se presenta una aplicación de tres capas compuesta por un frontend, un api, una database y un caché. El frontend se expone en un puerto HTTP, la API también puede exponerse para facilitar pruebas locales, y las relaciones `connects_to` permiten modelar la comunicación entre componentes. Este ejemplo ilustra cómo el lenguaje puede expresar una arquitectura relativamente realista en pocas líneas, delegando en el compilador la generación de una configuración más extensa y detallada

```
app MiAplicacion {  
  frontend web using "nginx:latest";  
  api backend using "node:20";  
  database db using "postgres:16";  
  cache redis using "redis:latest";  
  
  expose web on 8080;  
  expose backend on 3000;  
  
  web connects_to backend;  
  backend connects_to db;  
  backend connects_to redis;  
}
```

Código 6.3: Aplicación multicontenedor con frontend, API, base de datos y caché.